ProcureGuard AI

Agentic invoice exception resolution

Rishi Dwivedi - 24070126152
Siddarth Singh Kushwah - 24070126174
AIML-C, SIT Pune | CA-3 | Team of two

Problem and domain

In procurement and accounts payable, supplier invoices must agree with an authorised purchase order, received goods and trusted supplier details. A mismatch is only the beginning of an investigation: staff still have to locate a missing receipt, establish whether a price amendment was approved, or verify a bank change. Evidence arrives from different owners and may be incomplete, altered or contradictory. Repeated handoffs leave cases unresolved and make the final decision hard to reproduce.

Proposed solution

ProcureGuard is a Python and Streamlit proof of concept that investigates exceptions using synthetic operational data. A planner observes verified findings and selects a permitted source lookup. An evidence agent checks content hashes, order/vendor/item scope, effective dates and approval authority. Accepted evidence changes a local source snapshot, never the invoice. The matching and risk agents reassess the case, and a separately implemented critic recomputes the accounting findings before the decision coordinator recommends ready for human review, hold, or insufficient data.

Project differentiator

The focus is resolution after an exception: the next tool changes with the remaining evidence gap. A two-gap invoice needs both an approved amendment and a receipt; obtaining the amendment alone cannot clear the case. A delayed receipt can produce a new case revision after a reviewer requests a recheck. Changed or conflicting records remain blocked. Cumulative quantities prevent distinct invoices from reusing the same receipt capacity.

Saved investigations reopen after a browser restart without repeating tool execution. Simulated human lifecycle classification excludes inactive submissions from capacity and triggers revalidation across sessions. A real installed local model has also selected evidence tools in a recorded bounded demonstration.

Technology and evaluation

The implementation uses Python, Streamlit, Decimal and SQLite. An explicit offline mock planner ensures a free, reproducible demonstration; an optional Ollama adapter supports local structured tool selection. Models cannot bypass numerical checks or execute payments. Tests cover reconciliation, evidence trust, invalid planner actions, bounded loops, concurrent saves, stale reviews, audit tampering and UI flows. A rules-only baseline uses the same checker without evidence acquisition. Actual results are provided in docs/validation_report.md and reports/.

Scope and expected value

The intended benefits are consistent investigation, a clear next owner, and a reproducible evidence-backed handoff. No organisational savings are claimed. Sources and reviewer identities are simulated. The current scope is single-line INR records, without tax, credit notes, real ERP connections, production identity or payment capability. Future work includes signed source approvals, authenticated integrations, multi-line reconciliation and independent field evaluation.